```
"""
Video Auto Recap (Multi-Account) — REAL
working backend
----------------------------------------
--------------
- Upload a video
- Gemini API samples frames from the video
and writes a Burmese recap script
  in the chosen narrative tone
- edge-tts converts the script to Burmese
speech (male/female)
- FFmpeg builds the final video:
blurred/black background, correct aspect
  ratio, burned-in subtitles with keyword
highlighting, muxed with the new
  voice track
- No artificial daily limit, no PRO lock —
everything here is free-tier only.
"""

import os
import re
import json
import random
import shutil
import asyncio
import tempfile
import subprocess
from pathlib import Path
```

```
import gradio as gr

# ----------------------------------------
------------------------
# Optional deps: only imported when
actually needed so the app still
# boots even if a key/library is missing,
and gives a clear error.
# ----------------------------------------
------------------------
def _lazy_genai():
    import google.generativeai as genai
    return genai


def _lazy_edge_tts():
    import edge_tts
    return edge_tts


WORKDIR = Path(tempfile.gettempdir()) /
"video_auto_recap"
WORKDIR.mkdir(exist_ok=True)

BURMESE_VOICES = {
    "Female (မိန်းကလေး)": "my-MM-
NilarNeural",
    "Male (ယောကျ်ားလေး)": "my-MM-
ThihaNeural",
}

TONE_INSTRUCTIONS = {
    "Thriller": "ထိတ်လန့်ဖွယ်၊ အားလပ်ချိန်မရှိအောင်
စိတ်လှုပ်ရှားစေတဲ့၊ suspense ပြည့်ဝတဲ့ အသံအနေအထား
```

```
နဲ့",
    "Casual": "ရိုးရှင်း၊ နေ့စဉ်စကားပြောသလို ပေါ့ပေါ့ပါးပါး",
    "Dramatic": "ခံစားချက်ပြင်းထန်၊ အနုပညာဆန်တဲ့၊ emotional",
    "Comedy": "ဟာသဖြင့် ရယ်စရာကောင်းအောင်",
    "Documentary": "အချက်အလက်အခြေခံ၊ ဓမ္မဓိဋ္ဌာန်ကျတဲ့ narrator style",
}


#
============================================
=========================
# 1. Utility: probe video
#
============================================
=========================
def probe_duration(video_path: str) ->
float:
    out = subprocess.run(
        ["ffprobe", "-v", "error", "-
show_entries", "format=duration",
         "-of",
"default=noprint_wrappers=1:nokey=1",
video_path],
        capture_output=True, text=True
    )
    try:
        return float(out.stdout.strip())
    except ValueError:
        return 0.0
```

```
def extract_sample_frames(video_path: str,
out_dir: Path, n_frames: int = 6):
    """Grab n evenly-spaced frames so
Gemini can 'see' the video."""
    duration = probe_duration(video_path)
or 10
    out_dir.mkdir(parents=True,
exist_ok=True)
    paths = []
    for i in range(n_frames):
        t = max(0.2, duration * (i + 1) /
(n_frames + 1))
        frame_path = out_dir /
f"frame_{i:02d}.jpg"
        subprocess.run(
            ["ffmpeg", "-y", "-ss", str(t),
"-i", video_path,
             "-frames:v", "1", "-q:v", "3",
str(frame_path)],
            capture_output=True
        )
        if frame_path.exists():
            paths.append(frame_path)
    return paths


#
============================================
=========================
# 2. Gemini: generate Burmese recap script
#
============================================
=========================
def generate_script_with_gemini(video_path:
```

```
str, api_key: str, tone: str,

target_seconds: float) -> str:
    if not api_key:
        raise gr.Error("Gemini API Key ထည့်ပေးပါ — script auto-generate လုပ်ဖို့ လိုအပ်ပါတယ်။")

    genai = _lazy_genai()
    genai.configure(api_key=api_key)

    frame_dir = WORKDIR / "frames"
    if frame_dir.exists():
        shutil.rmtree(frame_dir)
    frames =
extract_sample_frames(video_path,
frame_dir, n_frames=6)
    if not frames:
        raise gr.Error("Video ထဲက frame တွေ ထုတ်လို့မရပါ — file format စစ်ပေးပါ။")

    model = genai.GenerativeModel("gemini-
2.0-flash")

    tone_desc = TONE_INSTRUCTIONS.get(tone,
TONE_INSTRUCTIONS["Casual"])
    words_target = max(40,
int(target_seconds * 2.6))  # ~2.6 Burmese
words/sec speech

    prompt = (
        "ဒီ frame photos တွေက video တစ်ခုထဲကနေ အစဉ်လိုက်ယူထားတဲ့ ပုံတွေဖြစ်ပါတယ်။ "
        f"ဒီ video အကြောင်း {tone_desc} နဲ့
```

```
မြန်မာဘာသာနဲ့ recap narration script "
        f"တစ်ခုရေးပေးပါ။ Script က စကားပြော
သံထွက်အတွက် ရေးထားရမှာမို့ "
        "subtitle line တစ်ကြောင်းစီကို ၅-၉ လုံး
အထိသာ ရှည်ပါစေ၊ စာကြောင်းချင်း "
        f"newline နဲ့ ပိုင်းခြားပါ။ စုစုပေါင်း
{words_target} လုံးဝန်းကျင် ရှိရမည်။ "
        "Markdown, ခေါင်းစဉ်, ရှင်းလင်းချက် တွေ့ လုံး
၀ မထည့်ပါနဲ့ — script စာသား "
        "ကိုသာ ပြန်ပေးပါ။"
    )

    content = [prompt]
    for f in frames:
        content.append({"mime_type":
"image/jpeg", "data": f.read_bytes()})

    response =
model.generate_content(content)
    script = (response.text or "").strip()
    if not script:
        raise gr.Error("Gemini က script မ
ထုတ်ပေးနိုင်ပါ — API key / quota ကို စစ်ပေးပါ။")
    return script


#
============================================
=========================
# 3. TTS: Burmese voice via edge-tts (free,
no key needed)
#
============================================
=========================
```

```
async def _tts_async(text: str, voice: str,
rate_percent: int, out_path: str):
    edge_tts = _lazy_edge_tts()
    rate_str = f"{'+' if rate_percent >= 0
else ''}{rate_percent}%"
    communicate =
edge_tts.Communicate(text, voice,
rate=rate_str)
    await communicate.save(out_path)


def text_to_speech(script: str,
voice_gender: str, sync_speed: float) ->
str:
    voice =
BURMESE_VOICES.get(voice_gender,
BURMESE_VOICES["Female (မိန်းကလေး)"])
    # sync_speed slider is 1.0-1.6 ->
convert to edge-tts rate % (0% .. +45%)
    rate_percent = int((sync_speed - 1.0) /
0.6 * 45)
    out_path = str(WORKDIR / "voice.mp3")
    asyncio.run(_tts_async(script, voice,
rate_percent, out_path))
    if not Path(out_path).exists():
        raise gr.Error("TTS voice generate
မလုပ်နိုင်ပါ — internet connection ကို စစ်ပေးပါ။")
    return out_path


#
==========================================
========================
# 4. Subtitle (.ass) builder with keyword
```

```
highlighting
#
==========================================
========================
def hex_to_ass_color(hex_color: str) ->
str:
    """#RRGGBB -> &H00BBGGRR (ASS uses BGR
order)."""
    hex_color = (hex_color or
"#FFFF00").lstrip("#")
    if len(hex_color) != 6:
        hex_color = "FFFF00"
    r, g, b = hex_color[0:2],
hex_color[2:4], hex_color[4:6]
    return f"&H00{b}{g}{r}".upper()


def build_ass_subtitles(script: str,
audio_duration: float, text_color: str,
                        outline_color:
str, subtitle_ratio: int,
                        keyword_prob: int,
keyword_width: int,
                        aspect_w: int,
aspect_h: int) -> str:
    lines = [l.strip() for l in
script.splitlines() if l.strip()]
    if not lines:
        lines = [script.strip() or "..."]

    n = len(lines)
    per_line = audio_duration / n if n else
audio_duration
```

```
    font_size = max(18, int(aspect_h *
(subtitle_ratio / 100) * 0.12))
    primary = hex_to_ass_color(text_color)
    outline =
hex_to_ass_color(outline_color)
    highlight = "&H0000AAFF"  # bright
orange highlight (BGR)

    def fmt_t(t):
        h = int(t // 3600)
        m = int((t % 3600) // 60)
        s = t % 60
        return f"{h:d}:{m:02d}:{s:05.2f}"

    header = f"""[Script Info]
ScriptType: v4.00+
PlayResX: {aspect_w}
PlayResY: {aspect_h}
ScaledBorderAndShadow: yes

[V4+ Styles]
Format: Name, Fontname, Fontsize,
PrimaryColour, SecondaryColour,
OutlineColour, BackColour, Bold, Italic,
Underline, StrikeOut, ScaleX, ScaleY,
Spacing, Angle, BorderStyle, Outline,
Shadow, Alignment, MarginL, MarginR,
MarginV, Encoding
Style: Default,Noto Sans Myanmar,
{font_size},{primary},{primary},
{outline},&H00000000,1,0,0,0,100,100,0,0,1,
3,1,2,40,40,{int(aspect_h*0.08)},1

[Events]
```

```
Format: Layer, Start, End, Style, Name,
MarginL, MarginR, MarginV, Effect, Text
"""

    events = []
    t = 0.0
    for line in lines:
        start, end = t, t + per_line
        words = line.split()
        out_words = []
        for w in words:
            if random.randint(1, 100) <=
keyword_prob:
                out_words.append(
                    f"
{{\\c{highlight}\\bord{max(1, keyword_width
// 30)}}}{w}{{\\c{primary}\\bord3}}"
                )
            else:
                out_words.append(w)
        text = " ".join(out_words)
        events.append(
            f"Dialogue: 0,{fmt_t(start)},
{fmt_t(end)},Default,,0,0,0,,{text}"
        )
        t = end

    ass_path = str(WORKDIR / "subs.ass")
    with open(ass_path, "w", encoding="utf-
8") as f:
        f.write(header + "\n".join(events))
    return ass_path
```

```
#
============================================
=========================
# 5. FFmpeg: compose final video
#
============================================
=========================
ASPECTS = {
    "9:16 (Tiktok/Reels)": (1080, 1920),
    "16:9 (YouTube)": (1920, 1080),
    "1:1 (Square)": (1080, 1080),
}


def build_final_video(video_path: str,
audio_path: str, ass_path: str,
                      aspect_label: str,
bg_style: str) -> str:
    w, h = ASPECTS.get(aspect_label, (1080,
1920))
    out_path = str(WORKDIR /
"final_output.mp4")
    ass_escaped = ass_path.replace("\\",
"/").replace(":", "\\:")

    if bg_style.startswith("Blur"):
        bg_chain = (
            f"[0:v]scale={w}:
{h}:force_original_aspect_ratio=increase,"
            f"crop={w}:
{h},gblur=sigma=25[bg];"
        )
    else:
        bg_chain = f"color=c=black:s=
```

```
{w}x{h}[bg];"

    filter_complex = (
        f"{bg_chain}"
        f"[0:v]scale={w}:
{h}:force_original_aspect_ratio=decrease[fg
];"
        f"[bg][fg]overlay=(W-w)/2:(H-
h)/2[merged];"
        f"
[merged]ass='{ass_escaped}'[outv]"
    )

    cmd = [
        "ffmpeg", "-y",
        "-i", video_path,
        "-i", audio_path,
        "-filter_complex", filter_complex,
        "-map", "[outv]", "-map", "1:a",
        "-c:v", "libx264", "-preset",
"veryfast", "-crf", "20",
        "-c:a", "aac", "-b:a", "160k",
        "-shortest",
        out_path,
    ]
    result = subprocess.run(cmd,
capture_output=True, text=True)
    if not Path(out_path).exists():
        raise gr.Error(f"FFmpeg processing
မအောင်မြင်ပါ:\n{result.stderr[-1500:]}")
    return out_path


#
```

```
# ============================================================================
# 6. Main pipeline
# ============================================================================
def run_pipeline(video_file, api_key, aspect_label, bg_style,
                 voice_gender, tone, text_color, outline_color,
                 subtitle_ratio, keyword_prob, keyword_width, sync_speed,
                 progress=gr.Progress()):
    if video_file is None:
        raise gr.Error("Video upload လုပ်ပါဦး။")

    video_path = video_file if isinstance(video_file, str) else video_file.name

    progress(0.05, desc="Video ကို စစ်ဆေးနေသည်...")
    duration = probe_duration(video_path)
    if duration <= 0:
        raise gr.Error("Video file ကို ဖတ်လို့မရပါ — mp4 file ဟုတ်မဟုတ် စစ်ပေးပါ။")

    progress(0.15, desc="🧠 Gemini AI နှင့် Burmese script ရေးနေသည်...")
    script = generate_script_with_gemini(video_path, api_key, tone, duration)
```

```
    progress(0.45, desc="🎙️ Burmese အသံ
ဖန်တီးနေသည်...")
    audio_path = text_to_speech(script,
voice_gender, sync_speed)
    audio_duration =
probe_duration(audio_path) or duration

    progress(0.65, desc="✍️ Subtitle
တည်ဆောက်နေသည်...")
    w, h = ASPECTS.get(aspect_label, (1080,
1920))
    ass_path = build_ass_subtitles(
        script, audio_duration, text_color,
outline_color,
        int(subtitle_ratio),
int(keyword_prob), int(keyword_width), w, h
    )

    progress(0.8, desc="🎬 Video ကို FFmpeg
ဖြင့် ပေါင်းစပ်နေသည်...")
    final_path =
build_final_video(video_path, audio_path,
ass_path, aspect_label, bg_style)

    progress(1.0, desc="✅ ပြီးပါပြီ!")
    return final_path, script


#
==========================================
========================
# 7. Gradio UI (mirrors the original mock-
up layout)
#
```

```
==========================================
==========================
CUSTOM_CSS = """
.gradio-container {background-color:#111318
!important;}
#generate-btn {background:#ff7a1a
!important; color:white !important; font-
weight:700 !important;}
"""

with gr.Blocks(title="Video Auto Recap
(Multi-Account)", css=CUSTOM_CSS,
theme=gr.themes.Base()) as demo:
    gr.Markdown("## ✨ Video Auto Recap
(Multi-Account) ✨")
    gr.Markdown("🎉 **Free & Unlimited:**
Video ဘယ်နှစ်ခုမဆို ကန့်သတ်မရှိ ထုတ်ယူနိုင်ပါသည်။")

    with gr.Row():
        with gr.Column():
            video_in = gr.Video(label="📽
ဗီဒီယို ထည့်ရန်")
            output_video =
gr.Video(label="✅ ပြီးပြည့်စုံသော ဗီဒီယို")
        with gr.Column():
            api_key = gr.Textbox(
                label="🔑 သင်၏ Gemini API
Key ထည့်ရန် (AI Studio မှ အခမဲ့ ရယူနိုင်သည်)",
                placeholder="AIzaSy...",
type="password"
            )
            aspect = gr.Dropdown(
                list(ASPECTS.keys()),
value="9:16 (Tiktok/Reels)",
```

```
                label="ဗီဒီယိုအမျိုးအစား"
            )
            bg_style = gr.Radio(
                ["Blur Background (အနောက်ခံ
ပါးမည်)", "Black Background (အမည်းရောင်ထား
မည်)"],
                value="Blur Background
(အနောက်ခံ ပါးမည်)",
                label="နောက်ခံ ဖြည့်စွက်မှု"
            )

    gr.Markdown("### 🗣️ အသံနှင့် စာတန်းထိုး ဆက်
တင်များ")
    with gr.Row():
        voice_gender =
gr.Radio(list(BURMESE_VOICES.keys()),

value="Female (မိန်းကလေး)", label="AI အသံ
အမျိုးအစား")
        tone =
gr.Dropdown(list(TONE_INSTRUCTIONS.keys()),
value="Thriller",

label="Narrative Tone")

    with gr.Row():
        text_color =
gr.ColorPicker(value="#FFFF00", label="စာလုံး
အရောင်")
        outline_color =
gr.ColorPicker(value="#000000",
label="အနားသတ်အရောင်")

    subtitle_ratio = gr.Slider(0, 100,
```

```
value=15, step=1, label="မြန်မာစာတန်း တည်နေရာ %")
    keyword_prob = gr.Slider(50, 100, value=75, step=1, label="📍 မူရင်းစာတန်းဖျောက်မည့်နေရာ %")
    keyword_width = gr.Slider(5, 151, value=51, step=1, label="🖍 မူရင်းစာတန်း ဝါးမည့်ပမာဏ")
    sync_speed = gr.Slider(1.0, 1.6, value=1.35, step=0.01, label="🗣🎬 အသံနှင့် ဗီဒီယို အရှိန်ညှိနှုန်း")

    generate_btn = gr.Button("🚀 Generate Video", elem_id="generate-btn", size="lg")
    script_out = gr.Textbox(label="ထုတ်ပေးသော Burmese Script (အတည်ပြုရန်)", lines=6)

    generate_btn.click(
        run_pipeline,
        inputs=[video_in, api_key, aspect, bg_style, voice_gender, tone,
                text_color, outline_color, subtitle_ratio, keyword_prob,
                keyword_width, sync_speed],
        outputs=[output_video, script_out],
    )

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 7860))

demo.queue(max_size=20).launch(server_name="0.0.0.0", server_port=port)
```